API Documentation

COP290-Assignment2

Arin Chandurkar - 2024CS50002 Keshav Navandar - 2024CS10441

March 22, 2026

Contents

auditController.ts **2**

authController.ts **2**

boardController.ts **4**

column(STATUS)controller.ts **5**

commentController.ts **6**

notificationController.ts **6**

projectController.ts **7**

taskController.ts **8**

teamController.ts **10**

workflowController.ts **11**

auditController.ts

1) getBoardAuditLogs

- **TYPE:** FETCH API
- **DESCRIPTION:** Takes a specific board ID and fetches the audit data regarding that particular board. It queries the audit table directly, which prevents the use of excessive parameters and improves code readability.

2) getTaskAuditLogs

- **TYPE:** FETCH API
- **DESCRIPTION:** Takes a specific task ID and retrieves its history from the audit log, subsequently updating the activity timeline for that given task.

---

authController.ts

A) Token set up

- **generateAccessTokens:** Utilizes the JWT package to take the login username and email ID, creating a hashed access token that is utilized by subsequent API calls for authorization.
- **generateRefreshToken:** Checks the user ID and generates a refresh token. As a design decision to enhance security, access tokens expire in 15 minutes to minimize risk if compromised, while refresh tokens remain valid for 7 days to maintain user sessions.

B) Cookies

- **setAccessTokenCookie:** Stores the access token in a cookie with the `httpOnly: true` flag enabled. This prevents client-side JavaScript from accessing the token, thereby mitigating Cross-Site Scripting (XSS) attacks.
- **setRefreshTokenCookie:** Stores the refresh token securely using the identical `httpOnly: true` configuration for the aforementioned security reasons.

C) APIs

1) register

- **TYPE:** POST API
- **DESCRIPTION:** Attempts to register a new user. It first verifies if the given user already exists in the database; if so, it returns an error. Otherwise, it inserts the user's data and assigns a newly generated refresh token to the database to ensure session functionality.

2) login

- **TYPE:** POST API
- **DESCRIPTION:** Authenticates user credentials. The API first verifies the existence of the email ID in the database. It returns specific error messages depending on whether the email is unregistered or the password is incorrect. Upon successful login, the user's refresh token is renewed, allowing for a continuous 7-day session before requiring re-authentication.

3) refresh

- **TYPE:** POST API
- **DESCRIPTION:** Validates a provided refresh token and its corresponding user ID against the database. Upon successful validation, it issues a new access token to the user.

4) logout

- **TYPE:** POST API
- **DESCRIPTION:** Handles user logout by deleting the active refresh token from the database and clearing the associated authentication cookies from the client.

5) getProfile

- **TYPE:** FETCH API
- **DESCRIPTION:** Fetches the currently authenticated user's profile details from the database, utilizing the user ID extracted from the decoded access token.

6) updatedProfilePicture

- **TYPE:** PUT API
- **DESCRIPTION:** Validates the uploaded avatar against a regular expression for Base64 format. It then updates the user's profile picture. Based on recent design iterations, the entire Base64 string is stored directly within the `profile_url` database field rather than utilizing a file pointer or external URL.

7) change password

- **TYPE:** PUT API
- **DESCRIPTION:** Facilitates password updates by validating the user's old password, hashing the new password, and securely updating the database entry.

8) updateProfile

- **TYPE:** PUT API
- **DESCRIPTION:** Allows users to update their personal core profile information, such as their first name, surname, and email address. It validates the uniqueness of the new email address before applying modifications.

---

boardController.ts

1) getBoards

- **TYPE:** FETCH API
- **DESCRIPTION:** Retrieves all boards associated with a specific project ID. It also aggregates and returns the total count of tasks within each board's summary.

2) createBoard

- **TYPE:** POST API
- **DESCRIPTION:** Inserts a new board into the database given a name and color (defaulting to blue). It initializes default Work-In-Progress (WIP) limits: NULL for 'TO-DO' and 'DONE', and limits of 3 and 1 for the 'In-Progress' and 'In-Review' statuses, respectively. Default status columns are then appended to the database.

3) deleteBoard

- **TYPE:** DELETE API
- **DESCRIPTION:** Permanently removes a specified board and its associated dependencies from the database.

4) getBoardProject

- **TYPE:** FETCH API
- **DESCRIPTION:** Identifies the parent project for a given board ID and retrieves the current user's role within that project to facilitate strict permission and authorization checks.

---

column(STATUS)controller.ts

1) getColumns(STATUS)

- **TYPE:** FETCH API
- **DESCRIPTION:** Retrieves all status columns for a specified board, sorted in ascending order based on their position index (Left to Right).

2) createColumn(status)

- **TYPE:** POST API
- **DESCRIPTION:** Creates a new status column with a specified name and an initial WIP limit defaulting to 3. As a design decision to simplify position indexing, new columns are appended to the end of the board by default, though they can be reordered subsequently.

3) renameColumn

- **TYPE:** PUT API
- **DESCRIPTION:** Updates the designated display name of an existing status column.

4) updateColumnLimit

- **TYPE:** PUT API
- **DESCRIPTION:** Modifies the Work-In-Progress (WIP) limit constraint for a specified status column.

5) reorderColumns

- **TYPE:** PUT API
- **DESCRIPTION:** Accepts an array of column IDs and updates their corresponding position values in the database, effectively reordering the columns on the board.

6) deleteColumn

- **TYPE:** DELETE API
- **DESCRIPTION:** Deletes a specified status column from the database.

---

commentController.ts

1) getComments

- **TYPE:** FETCH API
- **DESCRIPTION:** Retrieves all comments associated with a specific task ID, joined with user data to identify the authors. The data is ordered chronologically from oldest to newest to establish a clear conversational timeline.

2) addComment

- **TYPE:** POST API
- **DESCRIPTION:** Appends a new comment to a task. It parses the comment for '@surname' mentions using regex, triggering notifications for tagged users and the assigned task owner. Surnames were chosen as the unique identifier for mentions as a specific design decision.

3) editComment

- **TYPE:** PUT API
- **DESCRIPTION:** Allows a user to modify an existing comment. It strictly verifies authorization by ensuring the requester is the original author before applying changes, throwing an error otherwise.

4) deleteComment

- **TYPE:** DELETE API
- **DESCRIPTION:** Deletes a user's own comment and logs the deletion event in the audit trail to maintain transparency regarding removed content.

---

notificationController.ts

1) getNotifications

- **TYPE:** FETCH API
- **DESCRIPTION:** Retrieves all notifications for the currently authenticated user, sorted in descending order by timestamp to prioritize the most recent alerts.

2) markAsRead

- **TYPE:** PUT API
- **DESCRIPTION:** Updates the status of a specific notification to 'read', clearing the unread status indicator on the client side.

3) markAllAsRead

- **TYPE:** PUT API
- **DESCRIPTION:** Efficiently updates all unread notifications for the user to a 'read' state simultaneously.

4) deleteNotification

- **TYPE:** DELETE API
- **DESCRIPTION:** Permanently removes a specified notification record from the user's history.

5) createNotification

- **TYPE:** POST API
- **DESCRIPTION:** Manual endpoint to insert an alert or notification into a user's feed. It utilizes an internal sub-routine to structure and write the payload directly into the database.

---

projectController.ts

1) getProjects

- **TYPE:** FETCH API
- **DESCRIPTION:** Retrieves all active (non-archived) projects where the requesting user holds membership. This is primarily utilized to populate the main user dashboard.

2) getProjectById

- **TYPE:** FETCH API
- **DESCRIPTION:** Performs a comprehensive fetch of a specific project, including its constituent boards, assigned members, and the current user's specific role to facilitate granular permission checks.

3) getArchivedProjects

- **TYPE:** FETCH API
- **DESCRIPTION:** Fetches projects that have been marked as archived, removing them from the active dashboard view while retaining their data for administrative record-keeping.

4) createProject

- **TYPE:** POST API
- **DESCRIPTION:** Instantiates a new project. Based on authorization design decisions, this action is restricted to the global administrator (`admin@workspace.com`). Upon creation, the creator is automatically designated as the initial Project Admin.

5) updateProject

- **TYPE:** PUT API
- **DESCRIPTION:** Updates core project metadata, including the project name, description, current status, and progress indicators.

6) archiveProject

- **TYPE:** PUT API
- **DESCRIPTION:** Toggles the `is_archived` flag for a project. Project Admins utilize this function to declutter the workspace without executing permanent data deletion.

7) toggleStar

- **TYPE:** PUT API
- **DESCRIPTION:** Toggles a 'starred' status for a project, allowing users to pin favorite or high-priority projects to the top of their personalized dashboards.

8) deleteProject

- **TYPE:** DELETE API
- **DESCRIPTION:** Permanently removes a project and its associated data. To prevent accidental data loss, this destructive action is strictly limited to the global workspace administrator; standard project admins are restricted to the archiving function.

---

taskController.ts

1) getTasks

- **TYPE:** FETCH API
- **DESCRIPTION:** Retrieves all active (non-closed) tasks for a specified board, joined with user information to display task assignees.

2) createTask

- **TYPE:** POST API
- **DESCRIPTION:** Instantiates a new Task, Bug, or Story. If creating a sub-task, it validates that the parent task is of the 'Story' type. It verifies the WIP limits of the target column before permitting insertion and subsequently logs a 'CREATED' event in the audit trail.

3) moveTask

- **TYPE:** PUT API
- **DESCRIPTION:** Facilitates the core operational logic of the Kanban board by moving tasks between status columns. It rigorously checks WIP limits and validates transitions against defined "Allowed Workflows" (e.g., preventing backward movement or stage skipping). It also restricts the direct lateral movement of 'Story' type tasks.

4) updateTask

- **TYPE:** PUT API
- **DESCRIPTION:** Updates task attributes such as title, priority, and description. If the assignee is modified, it triggers a notification to the newly assigned user. Additionally, it enforces role-based access by restricting standard Project Members from assigning tasks to other individuals.

5) deleteTask

- **TYPE:** DELETE API
- **DESCRIPTION:** Permanently removes a task from the database. It also evaluates and re-synchronizes the status of the parent story if the deleted entity was a sub-task.

6) closeTask

- **TYPE:** PUT API
- **DESCRIPTION:** Transitions a task into a terminal 'Closed' state. To enforce proper workflow completion, this action is only permitted if the task resides in the final status column. It also records a final snapshot of the task closure in the audit log.

7) reSyncStories

- **TYPE:** PUT API
- **DESCRIPTION:** Triggers a resynchronization of all 'Story' type tasks on a specified board. It iterates through all stories on the board and updates each story's status strictly based on the progress of its sub-tasks.

8) getTaskById

- **TYPE:** FETCH API
- **DESCRIPTION:** Performs a detailed fetch for a single task using its unique ID. It retrieves comprehensive information by joining tables to return the task details, assignees, reporters, and the parent task information if applicable.

9) getChildTasks

- **TYPE:** FETCH API
- **DESCRIPTION:** Fetches all child tasks (sub-tasks) associated with a specific parent Story ID. It retrieves the sub-tasks sorted by newest first, joining with user data to display who is assigned to each sub-task.

---

teamController.ts

1) getMembers

- **TYPE:** FETCH API
- **DESCRIPTION:** Retrieves a list of all assigned members for a specific project, categorized by their distinct roles (e.g., Admins, Members, Viewers).

2) addMember

- **TYPE:** POST API
- **DESCRIPTION:** Invites a user to join a project via their registered email address. It validates whether the user exists in the system and ensures they are not already a project member. If a specific role is not provided, it defaults to 'Project Member'.

3) removeMember

- **TYPE:** DELETE API
- **DESCRIPTION:** Revokes a user's membership and access privileges for a specific project.

4) updateMemberRole

- **TYPE:** PUT API
- **DESCRIPTION:** Updates a member's role within a project. It enforces strict permission rules, ensuring global administrators' roles are protected, that regular project admins cannot change other project admins' roles, and that users cannot modify their own permissions.

5) getMyTeammates

- **TYPE:** FETCH API
- **DESCRIPTION:** Compiles a comprehensive directory of all co-workers and collaborators that share at least one project with the requesting user.

---

workflowController.ts

1) getProjectWorkflows

- **TYPE:** FETCH API
- **DESCRIPTION:** Retrieves all defined 'Allowed' transition rules between status columns for a specific project, establishing the permitted operational flow of the board.

2) addWorkflow

- **TYPE:** POST API
- **DESCRIPTION:** Defines a new valid state transition (e.g., moving from 'To Do' directly to 'In Progress'). It actively prevents duplicate entries to avoid conflicting rule definitions.

3) deleteWorkflow

- **TYPE:** DELETE API
- **DESCRIPTION:** Removes an existing workflow rule from the board, subsequently preventing users from executing that specific column transition.